



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	BE	Semester:	<u>VII</u>
Course Code:		Course Name:	R3

Name of Student:	BARI ANKIT VINOD
Roll No. :	61
Assignment No.:	3
Title of Assignment:	
Date of Submission:	
Date of Correction:	

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Completeness	5	5
Demonstrated Knowledge	3	2
Legibility	2	2
Total	10	5

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Completeness	5	3-4	1-2
Demonstrated Knowledge	3	2	1
Legibility	2	1	0

Checked by

Name of Faculty : Mrs. Ramesh Joshi

Signature :

Date :

Q. 1) An e-commerce platform uses a deep learning model for personalized product recommendations. However, the model is slow and consumes expensive computational resources. Apply optimization techniques to improve performance while maintaining recommendation quality.

→ (a) Model pruning - Remove redundant or low-important weight.

- How it helps - reduces model size, faster inference, lower memory usage.
- Types -
 - unstructured pruning (remove small weights)
 - structured pruning (remove neurons, channels)
- Impact - 20-70% reduction in parameters with minimal accuracy loss if fine-tuned properly.

(b) Quantization - Convert 32-bit floating point weight to lower precision.

- Techniques -
 - post-training quantization
 - quantization aware training
- benefits -
 - 2-4x faster inference
 - 50-75% memory reduction
- example tools - TensorRT, ONNX Runtime, PyTorch quantization.

(c) Knowledge distillation - Train a smaller "student" model to mimic a large "teacher" model.

∴ large model → Teacher ∴ small model → Student

- student learns → soft probability outputs
- internal representations.
- ∴ Result - much smaller model with near-original performance.

(d) data and feature optimization -

(i) feature selection - Remove low-importance features using SHAP analysis and feature importance scores.

less features → smaller input → faster inference

(ii) embedding optimization - embedding layers are usually the largest of recommendations systems.

optimized by - reducing embedding dimension, hashing trick, embed

Eg. Instead of 256-dim embeddings $\rightarrow$ use 64-dim.

(iii) candidate generation + Ranking split -

S1 - Candidate generation (fast, simple model)

- Matrix factorization and approximate nearest neighbors.

S2 - Ranking model (Heavy, precise)

- deep neural network

- This reduces full model computation from millions $\rightarrow$ few hundred items.

System-level optimization -

(a) Caching recommendations - popular items, recently viewed combinations, user embeddings.

Eg. Top-100 items per user cached in Redis.

(b) Batch inference - Instead of real-time inference for each request - process in batches, precompute recommendations periodically. Used in large systems like Alibaba.

(c) Hardware acceleration -

- GPU inference, TensorRT optimization, use CPUs with AVX instructions

If traffic is large - model serving via Triton inference server and Kubernetes autoscaling.

(d) Approximate nearest neighbor search

Use FAISS and HNSW, given $O(n) \rightarrow O(\log n)$

Q. 8) Given is a dataset of movies with their reviews given by users and ratings of top 250 movies. Based on the scenario apply a content-based recommendation system that uses posterior probability bayes rule -

→ S1 - Define Bayesian framework -

we compute - $P(\text{Like} | \text{Feature})$

$$P(\text{Like} | x) = \frac{P(x | \text{Like}) \cdot P(\text{Like})}{P(x)}$$

where,

x = movie features (genre, keyword, review code)

$P(\text{Like})$ → prior probability user likes movies

$P(x | \text{Like})$ = likelihood

$P(x)$ = evidence (normalization)

S2 - feature representation -

① genre encoding -

Eg. action → 1, drama → 0, comedy → 1

② review text → bag of words (TF-IDF)

Eg. "thrilling", "romantic", "slow"

S3 - define classes - we classify movie into -

• like (rating ≥ 4) • dislike (rating ≤ 4)

S4 - Compute probabilities -

$$P(\text{Like}) = \frac{\text{Number of liked movies}}{\text{Total movies}}$$

Eg.

If user liked 120 out of 200 ∴ $P(\text{Like}) = 120/200 = 0.6$

② Likelihood (Naive Bayes assumption) -

Assume feature independence -

$$P(x | \text{Like}) = \prod_i P(x_i | \text{Like})$$

Eg.

$$\hat{=} P(\text{Action} = 1 | \text{Like}) = 0.7$$

Use Laplace smoothing -

$$P(\text{word} | \text{Like}) = \frac{\text{count}(\text{word}, \text{like}) + 1}{\text{total words} + V}$$

SS: Recommendation algorithm -

1. Extract features
2. Compute - $P(\text{Like} | \text{Features})$ & $P(\text{Dislike} | \text{Features})$
3. choose higher probability -
4. Rank movies by probability -

(Code) -

```
from sklearn.feature_extraction.text import TfidfVectorizer  
from sklearn.naive_bayes import MultinomialNB  
from sklearn.model_selection import train_test_split
```

```
vectorizer = TfidfVectorizer(stop_words = 'english')
```

```
X_vec = vectorizer.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(X_vec, y, test_size=0.2)
```

```
model = MultinomialNB()
```

```
model.fit(X_train, y_train)
```

```
probs = model.predict_proba(X_test)
```

```
print("Probabilities = ", probs)
```